We consider the Auto.csv dataset that we have used in assignment 3. In this problem, we are interested in predicting the variable mpg = the recorded miles per gallon of the auto based on all the available independent variables which are listed below.

Cylinders = Number of cylinders between 4 and 8

Displacement = Engine displacement (cu. inches)

Horsepower = Engine horsepower

Weight = Vehicle weight (lbs.)

Acceleration = Time to accelerate from 0 to 60 mph (sec.)

Try to evaluate the models (model 1-5) that you constructed in assignment 3 following the steps below, and you are expected to write an analysis report which contains all the results from these steps.

Assignment 4
Analysis Report

1. Compare all the regression models that you constructed in assignment 3 by creating a comparison table with metrics: adjusted R-square, AIC, and BIC. What can you learn? Can we suggest the best model based on these metrics? Why?

<u>*Step 1: Construct a comparison table*</u>

Based on the assignment 3, we have the following OLS results for each model.

Model 1

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                    mpg   R-squared:                       0.760
Model:                            OLS   Adj. R-squared:                  0.755
Method:                 Least Squares   F-statistic:                     151.8
Date:                Tue, 17 Feb 2026   Prob (F-statistic):          9.04e-114
Time:                        10:49:47   Log-Likelihood:                -1081.3
No. Observations:                 392   AIC:                             2181.
Df Residuals:                     383   BIC:                             2216.
Df Model:                           8
Covariance Type:            nonrobust
==================================================================================
                     coef    std err          t      P>|t|      [0.025      0.975]
----------------------------------------------------------------------------------
const             62.5637      4.455     14.042      0.000      53.804      71.324
cylinders          0.7847      0.424      1.852      0.065      -0.048       1.618
displacement1     -0.0815      0.024     -3.356      0.001      -0.129      -0.034
displacement2      0.0001   4.35e-05      2.658      0.008    3.01e-05       0.000
horsepower1       -0.2969      0.053     -5.571      0.000      -0.402      -0.192
horsepower2        0.0007      0.000      3.984      0.000       0.000       0.001
weight1           -0.0021      0.003     -0.606      0.545      -0.009       0.005
weight2         9.793e-08   4.58e-07      0.214      0.831   -8.03e-07    9.99e-07
acceleration      -0.3866      0.133     -2.916      0.004      -0.647      -0.126
==============================================================================
Omnibus:                       39.429   Durbin-Watson:                   1.009
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               67.998
Skew:                           0.622   Prob(JB):                     1.72e-15
Kurtosis:                       4.618   Cond. No.                     2.52e+08
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 2.52e+08. This might indicate that there are
strong multicollinearity or other numerical problems.
```

Model 2

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                    mpg   R-squared:                       0.748
Model:                            OLS   Adj. R-squared:                  0.745
Method:                 Least Squares   F-statistic:                     229.3
Date:                Tue, 17 Feb 2026   Prob (F-statistic):          3.55e-113
Time:                        10:49:47   Log-Likelihood:                -1090.9
No. Observations:                 392   AIC:                             2194.
Df Residuals:                     386   BIC:                             2218.
Df Model:                           5
Covariance Type:            nonrobust
==================================================================================
                     coef    std err          t      P>|t|      [0.025      0.975]
----------------------------------------------------------------------------------
const            157.1480     12.362     12.712      0.000     132.843     181.453
cylinders          0.3237      0.355      0.912      0.362      -0.374       1.022
log displacement  -4.2379      1.616     -2.623      0.009      -7.415      -1.061
log_horsepower   -11.2428      1.919     -5.858      0.000     -15.016      -7.469
log_weight        -7.0777      2.759     -2.565      0.011     -12.503      -1.652
acceleration      -0.3750      0.131     -2.868      0.004      -0.632      -0.118
==============================================================================
Omnibus:                       39.957   Durbin-Watson:                   0.974
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               62.427
Skew:                           0.667   Prob(JB):                     2.78e-14
Kurtosis:                       4.430   Cond. No.                     1.25e+03
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.25e+03. This might indicate that there are
strong multicollinearity or other numerical problems.
```

Model 3

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                log_mpg   R-squared:                       0.791
Model:                            OLS   Adj. R-squared:                  0.789
Method:                 Least Squares   F-statistic:                     292.9
Date:                Tue, 17 Feb 2026   Prob (F-statistic):          6.18e-129
Time:                        10:49:47   Log-Likelihood:                 174.31
No. Observations:                 392   AIC:                            -336.6
Df Residuals:                     386   BIC:                            -312.8
Df Model:                           5
Covariance Type:            nonrobust
==================================================================================
                     coef    std err          t      P>|t|      [0.025      0.975]
----------------------------------------------------------------------------------
const              4.2203      0.098     42.956      0.000       4.027       4.413
cylinders         -0.0260      0.015     -1.718      0.087      -0.056       0.004
displacement -1.803e-05      0.000     -0.054      0.957      -0.001       0.001
horsepower        -0.0028      0.001     -4.513      0.000      -0.004      -0.002
weight            -0.0002   3.01e-05     -6.687      0.000      -0.000      -0.000
acceleration      -0.0057      0.005     -1.235      0.218      -0.015       0.003
==============================================================================
Omnibus:                        5.326   Durbin-Watson:                   0.890
Prob(Omnibus):                  0.070   Jarque-Bera (JB):                5.251
Skew:                           0.223   Prob(JB):                       0.0724
Kurtosis:                       3.351   Cond. No.                     3.87e+04
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 3.87e+04. This might indicate that there are
strong multicollinearity or other numerical problems.
```

Model 4

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                log_mpg   R-squared:                       0.805
Model:                            OLS   Adj. R-squared:                  0.803
Method:                 Least Squares   F-statistic:                     319.0
Date:                Tue, 17 Feb 2026   Prob (F-statistic):          1.20e-134
Time:                        10:49:47   Log-Likelihood:                 187.69
No. Observations:                 392   AIC:                            -363.4
Df Residuals:                     386   BIC:                            -339.6
Df Model:                           5
Covariance Type:            nonrobust
=====================================================================================
                        coef    std err          t      P>|t|      [0.025      0.975]
-------------------------------------------------------------------------------------
const                  8.7657      0.474     18.505      0.000       7.834       9.697
cylinders             -0.0206      0.014     -1.515      0.131      -0.047       0.006
log_displacement      -0.0901      0.062     -1.455      0.146      -0.212       0.032
log_horsepower        -0.4844      0.074     -6.588      0.000      -0.629      -0.340
log_weight            -0.3303      0.106     -3.123      0.002      -0.538      -0.122
acceleration          -0.0155      0.005     -3.100      0.002      -0.025      -0.006
==============================================================================
Omnibus:                        9.808   Durbin-Watson:                   0.989
Prob(Omnibus):                  0.007   Jarque-Bera (JB):               12.300
Skew:                           0.245   Prob(JB):                      0.00213
Kurtosis:                       3.717   Cond. No.                     1.25e+03
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.25e+03. This might indicate that there are
strong multicollinearity or other numerical problems.
```

Model 5

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.798
Model:                            OLS   Adj. R-squared:                  0.795
Method:                 Least Squares   F-statistic:                     305.0
Date:                Tue, 17 Feb 2026   Prob (F-statistic):          1.26e-131
Time:                        10:49:47   Log-Likelihood:                -50.812
No. Observations:                 392   AIC:                             113.6
Df Residuals:                     386   BIC:                             137.5
Df Model:                           5
Covariance Type:            nonrobust
=====================================================================================
                        coef    std err          t      P>|t|      [0.025      0.975]
-------------------------------------------------------------------------------------
const              14.5514      0.870     16.718      0.000      12.840      16.263
cylinders          -0.0254      0.025     -1.017      0.310      -0.075       0.024
log_displacement   -0.1980      0.114     -1.740      0.083      -0.422       0.026
log_horsepower     -0.8730      0.135     -6.461      0.000      -1.139      -0.607
log_weight         -0.5911      0.194     -3.042      0.003      -0.973      -0.209
acceleration       -0.0283      0.009     -3.071      0.002      -0.046      -0.010
==============================================================================
Omnibus:                       13.044   Durbin-Watson:                   0.979
Prob(Omnibus):                  0.001   Jarque-Bera (JB):               16.523
Skew:                           0.313   Prob(JB):                     0.000258
Kurtosis:                       3.787   Cond. No.                     1.25e+03
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.25e+03. This might indicate that there are
strong multicollinearity or other numerical problems.
```

	Model 1	**Model 2**	**Model 3**	**Model 4**	**Model 5**
Adjusted R-squared	0.755	0.745	0.789	0.803	0.795
AIC	2181	2194	-336.6	-363.4	113.6
BIC	2216	2218	-312.8	-339.6	137.5

Step 2: Comparing the result

Among the 5 models, Model 4 performs best based on those metric indicators.

It has the highest adjust R-squared (0.803), indicating the strongest explanatory power after transforming for the model complexity. It also has the lowest AIC (-363.4) and BIC (-339.6), suggesting it balances best between the likelihood of the model (goodness of fit) and the number of parameters (model complexity).

However, we can easily notice that the AIC and BIC of Model 1 & 2 (around 2200) have different range value than Model 3 & 4 (around -330 to -365), leading to the cause of variety in response transformations. Since Model 3 & 4 use the log transformation on the response value. In general, even with comparable groups, the Model 4 is still the best transformation model.

After the comparison, we learnt that:

- The transformation of the response value improves the model significantly.
- The combination of log response and log predictors (Model 4) gives the strongest performance.
- Box-Cox (Model 5) does not outperform simple log transformation.

According to the assignment 3, Model 4 also satisfies most of the regressions assumptions and has significantly interpretable predictors, contributing to the conclusion about the best performance model.

2. Use training-testing validation to evaluate the performance of the models. Specifically, randomly split the data into 80% training and 20% validation sample. Build the linear regression models with different model correction strategies that we used in assignment 3 using training sample, and predict on the validation set. Then assess the prediction accuracy. Repeat the entire procedure (data splitting, model construction, validation set prediction) 100 times. Report the average prediction mean squared error for each model. What can you learn from these results?

Step 1: Construct a dataframe for all models

```
#creating a dataframe for constructing all five models
```

```
poly = PolynomialFeatures(degree=2)
displacement = poly.fit_transform(data.iloc[:,2:3])
displacement = pd.DataFrame(displacement[:,1:], columns = ['displacement1','displacement2'])
horsepower = poly.fit_transform(data.iloc[:,3:4])
horsepower = pd.DataFrame(horsepower[:,1:], columns = ["horsepower1", "horsepower2"])
weight = poly.fit_transform(data.iloc[:,4:5])
weight = pd.DataFrame(weight[:,1:], columns = ["weight1", "weight2"])

log_displacement = np.log(data.iloc[:,2:3])
log_displacement = log_displacement.rename(columns={"displacement":"log_displacement"})
log_horsepower = np.log(data.iloc[:,3:4])
log_horsepower = log_horsepower.rename(columns={"horsepower":"log_horsepower"})
log_weight = np.log(data.iloc[:,4:5])
log_weight = log_weight.rename(columns={"weight":"log_weight"})
log_mpg = np.log(data.iloc[:,0:1])
log_mpg = log_mpg.rename(columns={"mpg":"log_mpg"})

Z, fit_lambda = stats.boxcox(data5.mpg)
Z = pd.DataFrame(Z, columns=['Z'])

data_new = pd.concat([Z,log_mpg,data.iloc[:,0:2],displacement,horsepower,weight,log_displacement,log_horsepower,log_weight,data.iloc[:,5]], axis = 1)
data_new
```

	Z	log_mpg	mpg	cylinders	displacement1	displacement2	horsepower1	horsepower2	weight1	weight2	log_displacement	log_horsepower	log_weight	acceleration
0	3.859148	2.890372	18.0	8	307.0	94249.0	130.0	16900.0	3504.0	12278016.0	5.726848	4.867534	8.161660	12.0
1	3.547736	2.708050	15.0	8	350.0	122500.0	165.0	27225.0	3693.0	13638249.0	5.857933	5.105945	8.214194	11.5
2	3.859148	2.890372	18.0	8	318.0	101124.0	150.0	22500.0	3436.0	11806096.0	5.762051	5.010635	8.142063	11.0
3	3.656731	2.772589	16.0	8	304.0	92416.0	150.0	22500.0	3433.0	11785489.0	5.717028	5.010635	8.141190	12.0
4	3.760349	2.833213	17.0	8	302.0	91204.0	140.0	19600.0	3449.0	11895601.0	5.710427	4.941642	8.145840	10.5
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
387	4.591882	3.295837	27.0	4	140.0	19600.0	86.0	7396.0	2790.0	7784100.0	4.941642	4.454347	7.933797	15.6
388	5.553254	3.784190	44.0	4	97.0	9409.0	52.0	2704.0	2130.0	4536900.0	4.574711	3.951244	7.663877	24.6
389	4.916211	3.465736	32.0	4	135.0	18225.0	84.0	7056.0	2295.0	5267025.0	4.905275	4.430817	7.738488	11.6
390	4.660422	3.332205	28.0	4	120.0	14400.0	79.0	6241.0	2625.0	6890625.0	4.787492	4.369448	7.872836	18.6
391	4.854801	3.433987	31.0	4	119.0	14161.0	82.0	6724.0	2720.0	7398400.0	4.779123	4.406719	7.908387	19.4

Step 2: Build the linear regression models with different model correction strategies, using the training data to predict on the validation set. Repeat the process 100.

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
error1 = []
error2 = []
error3 = []
error4 = []
error5 = []
for i in range(1,101):
    x = data_new.iloc[:,3:]
    y = data_new.iloc[:,:3]
    x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.20)

    ols = sm.OLS(y_train[["mpg"]],sm.add_constant(x_train[["cylinders","displacement1","displacement2",
                                                          "horsepower1","horsepower2","weight1","weight2","acceleration"]]))
    lm1 = ols.fit()
    y_pred1 = lm1.predict(sm.add_constant(x_test[["cylinders","displacement1","displacement2","horsepower1",
                                                  "horsepower2","weight1","weight2","acceleration"]]))
    error1.append(mean_squared_error(y_pred1,y_test[["mpg"]]))

    ols = sm.OLS(y_train[["mpg"]], sm.add_constant(x_train[["cylinders","log_displacement","log_horsepower","log_weight","acceleration"]]))
    lm2 = ols.fit()
    y_pred2 = lm2.predict(sm.add_constant(x_test[["cylinders","log_displacement","log_horsepower","log_weight","acceleration"]]))
    error2.append(mean_squared_error(y_pred2,y_test[["mpg"]]))

    ols = sm.OLS(y_train[["log_mpg"]], sm.add_constant(x_train[["cylinders","displacement1","horsepower1","weight1","acceleration"]]))
    lm3 = ols.fit()
    y_pred3 = np.exp(lm3.predict(sm.add_constant(x_test[["cylinders","displacement1","horsepower1","weight1","acceleration"]])))
    error3.append(mean_squared_error(y_pred3,y_test[["mpg"]]))

    ols = sm.OLS(y_train[["log_mpg"]], sm.add_constant(x_train[["cylinders","log_displacement","log_horsepower","log_weight","acceleration"]]))
    lm4 = ols.fit()
    y_pred4 = np.exp(lm4.predict(sm.add_constant(x_test[["cylinders","log_displacement","log_horsepower","log_weight","acceleration"]])))
    error4.append(mean_squared_error(y_pred4,y_test[["mpg"]]))

    ols = sm.OLS(y_train[["Z"]], sm.add_constant(x_train[["cylinders","log_displacement","log_horsepower","log_weight","acceleration"]]))
    lm5 = ols.fit()
    y_pred5 = (lm5.predict(sm.add_constant(x_test[["cylinders","log_displacement","log_horsepower",
                                                   "log_weight","acceleration"]]))*fit_lambda+1)**(1/fit_lambda)
    error5.append(mean_squared_error(y_pred5,y_test[["mpg"]]))
```

Step 3: Calculate mean of each error set and display

```
print(f"Mean of error 1: ",np.mean(error1))
print(f"Mean of error 2: ",np.mean(error2))
print(f"Mean of error 3: ",np.mean(error3))
print(f"Mean of error 4: ",np.mean(error4))
print(f"Mean of error 5: ",np.mean(error5))
```

```
Mean of error 1:  14.975602599588123
Mean of error 2:  15.571427391957053
Mean of error 3:  16.142139781937416
Mean of error 4:  15.30627106679999
Mean of error 5:  15.215653002376971
```

Step 4: Conclusion

Based on this result, the model one has the lowest mean squared error (14.98), suggesting it is slightly better in performing prediction on the unseen data. However, the difference between MSE score is very small which can indicate that all the model can predict the unseen data similarly.

3. Use 10-fold cross validation to evaluate the performance of the models. Report the average cross validation error for each model. What can you learn from these results?

Step 1: Construct 10-fold cross validation of the models

```
from sklearn.model_selection import KFold
cv = KFold(n_splits = 10, random_state = 1, shuffle = True)
x = data_new.iloc[:,3:]
y = data_new.iloc[:,:3]
error1 = []
error2 = []
error3 = []
error4 = []
error5 = []
for train_index, test_index in cv.split(data_new):
    x_train, x_test = x.iloc[train_index], x.iloc[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]

    ols = sm.OLS(y_train[["mpg"]],sm.add_constant(x_train[["cylinders","displacement1","displacement2",
                                                          "horsepower1","horsepower2","weight1","weight2","acceleration"]]))
    lm1 = ols.fit()
    y_pred1 = lm1.predict(sm.add_constant(x_test[["cylinders","displacement1","displacement2","horsepower1",
                                                 "horsepower2","weight1","weight2","acceleration"]]))
    error1.append(mean_squared_error(y_pred1,y_test[["mpg"]]))

    ols = sm.OLS(y_train[["mpg"]], sm.add_constant(x_train[["cylinders","log_displacement","log_horsepower","log_weight","acceleration"]]))
    lm2 = ols.fit()
    y_pred2 = lm2.predict(sm.add_constant(x_test[["cylinders","log_displacement","log_horsepower","log_weight","acceleration"]]))
    error2.append(mean_squared_error(y_pred2,y_test[["mpg"]]))

    ols = sm.OLS(y_train[["log_mpg"]], sm.add_constant(x_train[["cylinders","displacement1","horsepower1","weight1","acceleration"]]))
    lm3 = ols.fit()
    y_pred3 = np.exp(lm3.predict(sm.add_constant(x_test[["cylinders","displacement1","horsepower1","weight1","acceleration"]])))
    error3.append(mean_squared_error(y_pred3,y_test[["mpg"]]))
```


```
    ols = sm.OLS(y_train[["log_mpg"]], sm.add_constant(x_train[["cylinders","displacement1","horsepower1","weight1","acceleration"]]))
    lm3 = ols.fit()
    y_pred3 = np.exp(lm3.predict(sm.add_constant(x_test[["cylinders","displacement1","horsepower1","weight1","acceleration"]])))
    error3.append(mean_squared_error(y_pred3,y_test[["mpg"]]))
```


```
    ols = sm.OLS(y_train[["log_mpg"]], sm.add_constant(x_train[["cylinders","log_displacement","log_horsepower","log_weight","acceleration"]]))
    lm4 = ols.fit()
    y_pred4 = np.exp(lm4.predict(sm.add_constant(x_test[["cylinders","log_displacement","log_horsepower","log_weight","acceleration"]])))
    error4.append(mean_squared_error(y_pred4,y_test[["mpg"]]))

    ols = sm.OLS(y_train[["Z"]], sm.add_constant(x_train[["cylinders","log_displacement","log_horsepower","log_weight","acceleration"]]))
    lm5 = ols.fit()
    y_pred5 = (lm5.predict(sm.add_constant(x_test[["cylinders","log_displacement","log_horsepower",
                                                  "log_weight","acceleration"]]))*fit_lambda+1)**(1/fit_lambda)
    error5.append(mean_squared_error(y_pred5,y_test[["mpg"]]))
```

Step 2: Calculate mean of each error set and display

```
print(f"Mean of error 1: ",np.mean(error1))
print(f"Mean of error 2: ",np.mean(error2))
print(f"Mean of error 3: ",np.mean(error3))
print(f"Mean of error 4: ",np.mean(error4))
print(f"Mean of error 5: ",np.mean(error5))
```

```
Mean of error 1:  15.356105769216631
Mean of error 2:  16.022060171181995
Mean of error 3:  16.63868312874107
Mean of error 4:  15.769550358079155
Mean of error 5:  15.67861663763899
```

Step 3: Conclusion

Based on the mean values of each model, the Model 1 has the lowest mean squared value (15.35), which indicates the model 1 performs on predicting validation data best compared to other models. However, the difference in cross-validation error across models are relatively small. Therefore, we can suggest that all the models predict similarly.